Home Credit Default Risk: EDA

AUTHOR
Jake Hoopes

1 Setup

```
library(tidyverse)
theme_set(theme_bw(base_size = 11))

train <- read_csv("data/application_train.csv", show_col_types = FALSE)
test  <- read_csv("data/application_test.csv", show_col_types = FALSE)
bureau <- read_csv("data/bureau.csv", show_col_types = FALSE)

dim(train)
```

```
[1] 307511    122
```

```
dim(test)
```

```
[1] 48744   121
```

```
dim(bureau)
```

```
[1] 1716428      17
```

2 Introduction

2.1 Business Problem

Home Credit lends to people traditional banks turn away, and has done so since 1997. It operates in nine countries, has served more than 100 million customers, and reviews over 73 million applications a year. Because it targets high growth, low penetration markets on purpose, most applicants arrive with little or no credit history. Underwriting therefore runs on incomplete information, and that produces two expensive mistakes at scale. Approving someone who defaults drives up credit losses. Rejecting someone who would have repaid costs revenue and pushes away a creditworthy borrower, which cuts against the responsible lending reputation the company has built. Home Credit already holds better information than it currently uses, including external credit bureau history and internal records of past loans, balances, and payment behavior, but none of it is pulled together into a single risk score at the moment a decision is made. This project builds that score. Success means a lower default rate at the same approval rate, or a higher approval rate at the same default rate, measured with ROC AUC because only about 8% of historical loans show repayment trouble. This notebook covers the data understanding

phase that comes first: what is in the data, what is wrong with it, and which relationships look strong enough to build on.

2.2 Questions

1. How much of the predictive signal sits in the three external credit scores, and would a model without them still be useful?
2. What share of applicants have no credit bureau record, and do they default at a different rate than applicants with history?
3. Is reported income believable, and does payment-to-income separate defaulters from non-defaulters?
4. How often does the loan amount exceed the price of the goods financed, and does loan-to-value relate to default?
5. What does the `DAYS_EMPLOYED` value of 365243 represent, and should that group be modeled separately?
6. Is missingness itself predictive? Do applicants with missing building or vehicle information default at different rates?
7. Which application characteristics show the largest differences in default rate, and are those segments large enough to act on?
8. Does default risk change smoothly with age and employment tenure, or are there breaks that argue for binning?
9. Do recent credit bureau inquiries relate to default, and over what lookback window?

2.3 Key Findings at a Glance

The full evidence is in the sections below, and the Results and Conclusions section collects every decision. In brief:

- **Default is rare.** Only 8.07% of applicants had payment difficulties, so accuracy is a misleading metric and ROC AUC is used instead.
- **External credit scores carry the strongest signal.** Default runs from 3.2% to 20.0% across `EXT_SOURCE_3` deciles, but `EXT_SOURCE_1` is missing for 56.4% of applicants and none of the scores has a documented derivation.
- **No credit history means higher risk.** The 14.3% of applicants with no bureau record default at 10.12%, against 7.73% for those with a history. The depth of history is U-shaped rather than linear.
- **Missing information is itself predictive.** Applicants who left 51 or more fields blank default at 11.2%, against 6.1% for complete records.
- **One "impossible" value is a category.** `DAYS_EMPLOYED = 365243` marks pensioners, who default less often than everyone else.
- **Affordability barely registers.** Reported income correlates with default at -0.004, and payment-to-income is nearly as weak.
- **Age is a strong predictor and a protected characteristic.** It needs a fairness review before it is used in any model.

These are associations in historical data, not causes of default.

3 Description of the Data

```
# Each application file holds one row per loan application. bureau holds one
# row per prior credit reported to the credit bureau, so it runs many rows per
# applicant. Checking each key for duplicates is what confirms the grain.
tibble(
  table      = c("application_train", "application_test", "bureau"),
  rows       = c(nrow(train), nrow(test), nrow(bureau)),
  columns    = c(ncol(train), ncol(test), ncol(bureau)),
  key        = c("SK_ID_CURR", "SK_ID_CURR", "SK_ID_BUREAU"),
  key_unique = c(!any(duplicated(train$SK_ID_CURR)),
                 !any(duplicated(test$SK_ID_CURR)),
                 !any(duplicated(bureau$SK_ID_BUREAU)))
) |>
  knitr::kable(caption = "Grain, key, and size of each table used")
```

Grain, key, and size of each table used

table	rows	columns	key	key_unique
application_train	307511	122	SK_ID_CURR	TRUE
application_test	48744	121	SK_ID_CURR	TRUE
bureau	1716428	17	SK_ID_BUREAU	TRUE

The grain of `application_train` and `application_test` is one row per loan application, keyed by `SK_ID_CURR`, and that key is unique in both. The two files split the same population: `application_test` carries all 122 columns except `TARGET`, which is the column being predicted.

3.1 Column groups

```
# Assign every column to a group based on its name. The order of these rules
# matters, because a later rule only sees columns no earlier rule claimed.
col_groups <- tibble(column = names(train)) |>
  mutate(group = case_when(
    column %in% c("SK_ID_CURR", "TARGET") ~ "Identifier and target",
    column %in% c("NAME_CONTRACT_TYPE", "AMT_INCOME_TOTAL", "AMT_CREDIT",
                  "AMT_ANNUITY", "AMT_GOODS_PRICE") ~ "Loan contract and amounts",
    str_detect(column, "^EXT_SOURCE_") ~ "External credit scores",
    column %in% c("CODE_GENDER", "CNT_CHILDREN", "CNT_FAM_MEMBERS", "DAYS_BIRTH",
                  "NAME_TYPE_SUITE", "NAME_INCOME_TYPE", "NAME_EDUCATION_TYPE",
                  "NAME_FAMILY_STATUS", "NAME_HOUSING_TYPE", "FLAG_OWN_CAR",
                  "FLAG_OWN_REALTY", "OWN_CAR_AGE") ~ "Client demographics",
    column %in% c("DAYS_EMPLOYED", "OCCUPATION_TYPE",
                  "ORGANIZATION_TYPE") ~ "Employment",
    str_detect(column, "^(FLAG_|DAYS_LAST_PHONE_CHANGE)") ~ "Contact and document flags",
    str_detect(column, "^(REGION_|REG_|LIVE_)") ~ "Region and address",
```

```
    str_detect(column, "^(WEEKDAY_APPR|HOUR_APPR|DAYS_REGISTRATION|DAYS_ID_PUBLISH)") ~
        "Application timing",
    str_detect(column, "(_AVG|_MODE|_MEDI)$") ~ "Building information",
    str_detect(column, "_CNT_SOCIAL_CIRCLE$") ~ "Social circle",
    str_detect(column, "^AMT_REQ_CREDIT_BUREAU") ~ "Credit bureau inquiries",
    TRUE ~ "Unassigned"
  ))

col_groups |>
  count(group, sort = TRUE, name = "columns") |>
  knitr::kable(caption = "The 122 application columns by group")
```

The 122 application columns by group

group	columns
Building information	47
Contact and document flags	27
Client demographics	12
Region and address	9
Credit bureau inquiries	6
Loan contract and amounts	5
Application timing	4
Social circle	4
Employment	3
External credit scores	3
Identifier and target	2

The 122 columns are not 122 independent pieces of information. **Building information alone accounts for 47 of them**, and those 47 describe the apartment block the client lives in rather than the client. Contact and document flags account for another 27, most of which are binary indicators of whether a numbered document was supplied. Only five columns describe the loan itself, and only three carry external credit scores. So the file is far narrower than its width suggests, which matters for how much genuinely new signal there is to find.

3.2 Bureau join path and coverage

```
# bureau joins to the application tables on SK_ID_CURR, many bureau rows to one
# applicant. The share of applicants with no bureau row at all is the
# thin-file population the business problem is built around.
train |>
  mutate(has_bureau_record = SK_ID_CURR %in% bureau$SK_ID_CURR) |>
  count(has_bureau_record, name = "applicants") |>
  mutate(share = applicants / sum(applicants)) |>
  knitr::kable(digits = 3, caption = "Train applicants with at least one bureau record")
```

Train applicants with at least one bureau record

has_bureau_record	applicants	share
FALSE	44020	0.143
TRUE	263491	0.857

```
# Among applicants who do appear in bureau, how deep is their history?
bureau |>
  count(SK_ID_CURR, name = "credits") |>
  summarise(median = median(credits),
            mean   = round(mean(credits), 2),
            max    = max(credits)) |>
  knitr::kable(caption = "Bureau credits per applicant")
```

Bureau credits per applicant

median	mean	max
4	5.61	116

The join path is `application.SK_ID_CURR` to `bureau.SK_ID_CURR`, one to many. **14.3% of training applicants have no bureau record at all**, and the applicants who do appear carry a median of 4 prior credits. Any feature built from bureau therefore has to define what it means when there is nothing to aggregate, because a missing bureau average is not the same thing as an average of zero.

3.3 Discrepancies between the dictionary and the data

```
# The dictionary names the bureau key SK_BUREAU_ID. Check both spellings
# against the actual column names.
c("SK_BUREAU_ID", "SK_ID_BUREAU") %in% names(bureau)
```

```
[1] FALSE  TRUE
```

Four problems with the supplied dictionary are worth recording before relying on it:

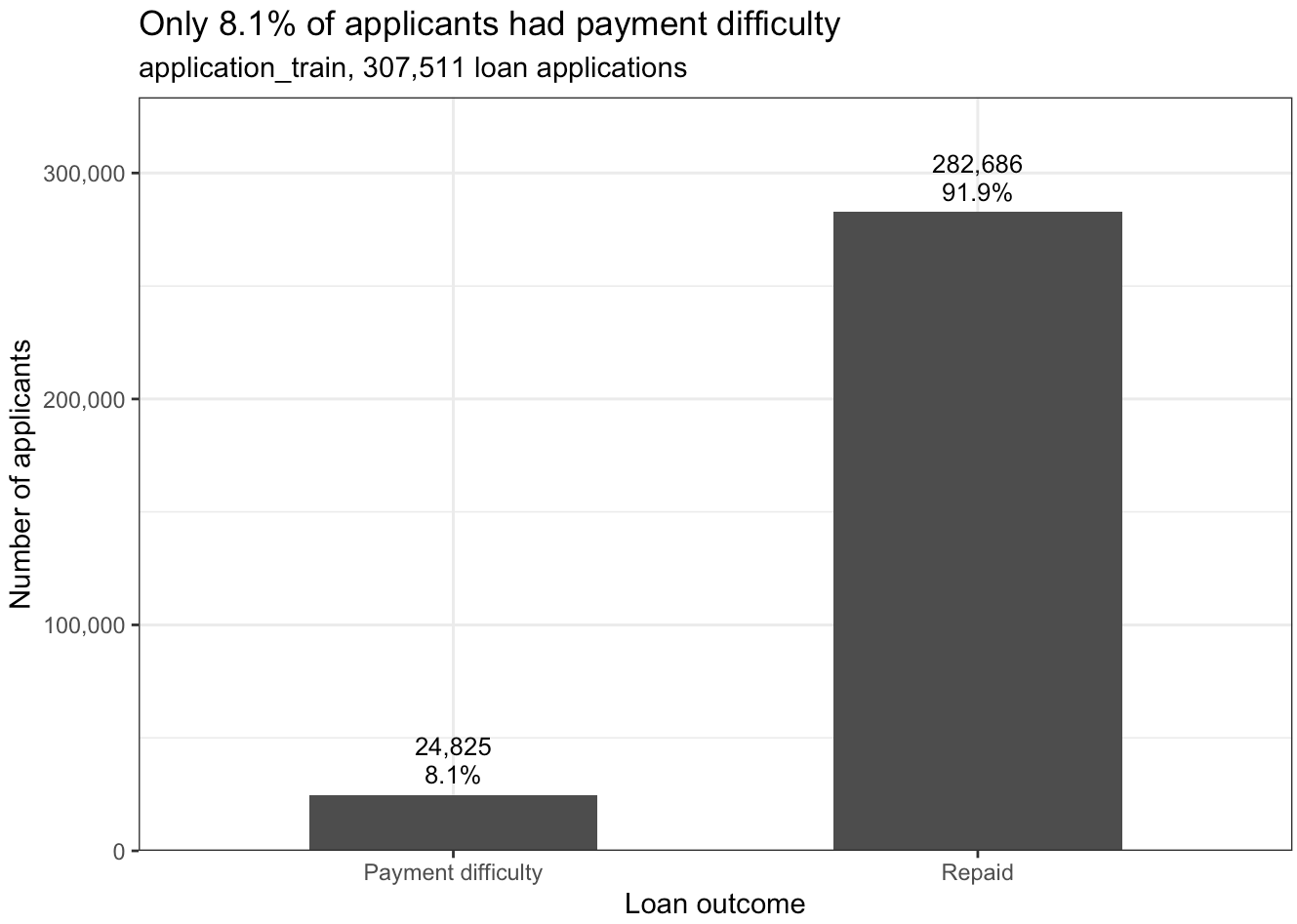
1. **The bureau key is misnamed.** The dictionary calls it `SK_BUREAU_ID`; the actual column is `SK_ID_BUREAU`, as the check above confirms. Code written from the dictionary fails on contact.
2. **The dictionary's row numbering skips 3 and 4**, so the file is visibly incomplete even though all 122 application columns happen to be documented.
3. `FLAG_WORK_PHONE` **and** `FLAG_PHONE` **carry identical descriptions**, both reading "Did client provide home phone." One is wrong and the dictionary gives no way to tell which.
4. **All 47 building columns share one copy-pasted description.** No individual definition exists for any of them, so the difference between `_AVG`, `_MODE`, and `_MEDI` has to be inferred from the data rather than read.

The practical consequence is that the dictionary is a starting point, not an authority. Every column used in this analysis gets verified against the data itself.

4 Data Integrity

4.1 Target distribution

```
# TARGET is 1 when the client had payment difficulty and 0 otherwise. Plotting
# the raw counts alongside the share shows how rare the positive class is.
train |>
  count(TARGET) |>
  mutate(share  = n / sum(n),
         status = if_else(TARGET == 1, "Payment difficulty", "Repaid"),
         label  = paste0(scales::comma(n), "\n", scales::percent(share, accuracy = 0.1)))
        |>
  ggplot(aes(x = status, y = n)) +
  geom_col(width = 0.55, fill = "grey30") +
  geom_text(aes(label = label), vjust = -0.25, size = 3.4, lineheight = 0.95) +
  scale_y_continuous(labels = scales::comma,
                     expand = expansion(mult = c(0, 0.18))) +
  labs(title = "Only 8.1% of applicants had payment difficulty",
       subtitle = "application_train, 307,511 loan applications",
       x = "Loan outcome", y = "Number of applicants")
```



The classes are heavily imbalanced at roughly 11 to 1. This confirms the assumption in the business problem statement and it drives two decisions. First, accuracy is useless as a metric here, since predicting “repaid” for everyone scores 91.9% while identifying nobody. ROC AUC stays the metric. Second, any model will need either class weighting or a tuned decision threshold, because the default 0.5 cutoff on a base rate this low will almost never fire.

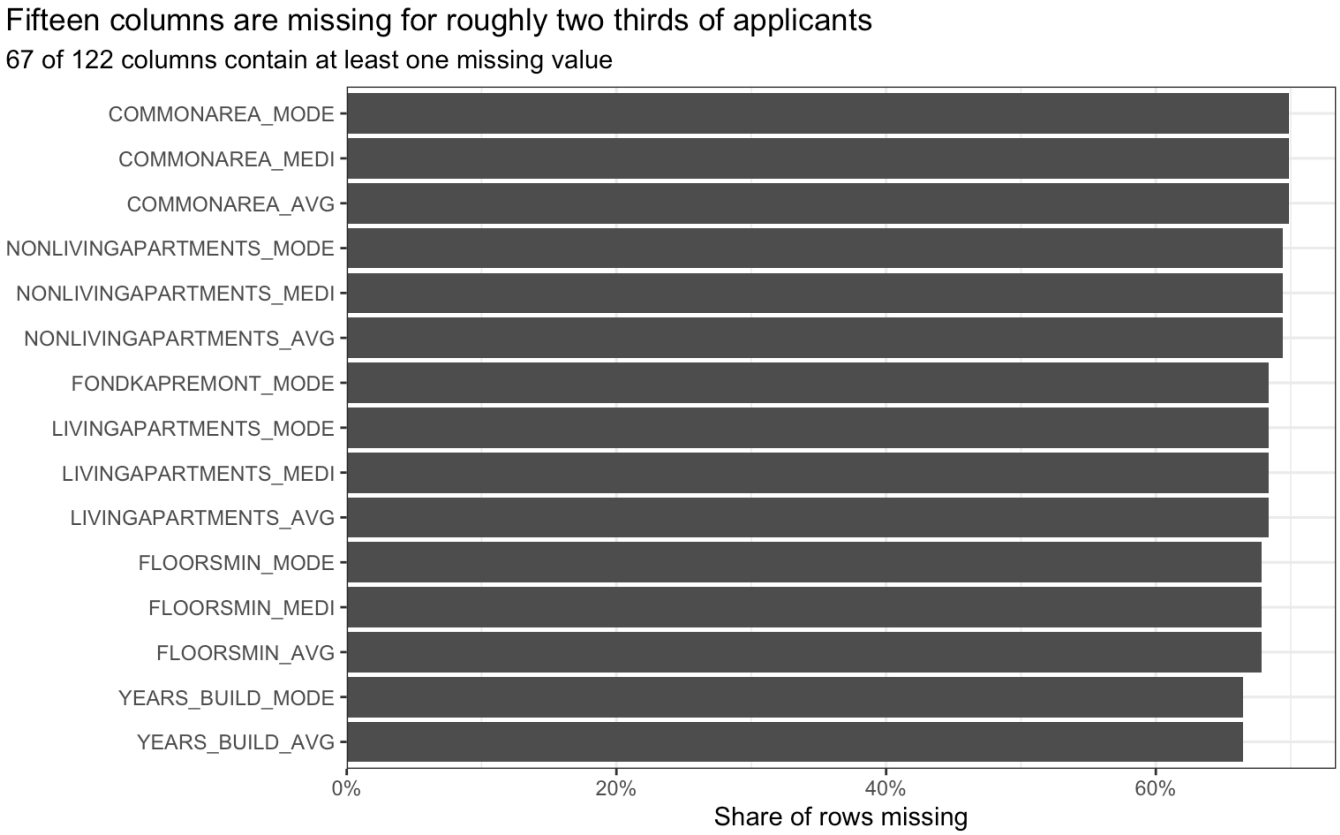
4.2 Missingness and its relation to the target

```
# Compute the share of missing values for every column, then keep only the
# columns that actually have gaps.
missing_by_col <- train |>
  summarise(across(everything(), ~ mean(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "column", values_to = "pct_missing") |>
  filter(pct_missing > 0) |>
  arrange(desc(pct_missing))

nrow(missing_by_col)
```

```
[1] 67
```

```
# Plot the 15 worst offenders. fct_reorder sorts the bars by size rather than
# alphabetically, which is what makes a horizontal bar chart readable.
missing_by_col |>
  slice_head(n = 15) |>
  ggplot(aes(x = pct_missing, y = fct_reorder(column, pct_missing))) +
  geom_col(fill = "grey30") +
  scale_x_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.05))) +
  labs(title = "Fifteen columns are missing for roughly two thirds of applicants",
       subtitle = "67 of 122 columns contain at least one missing value",
       x = "Share of rows missing", y = NULL) +
  # Align the title with the whole plot so the long column labels don't push it off the
       edge.
  theme(plot.title.position = "plot")
```




```
# Missingness matters most if it is informative. For a handful of columns,
# compare the default rate of rows where the value is missing against rows
# where it is present.
missing_vs_target <- function(col) {
  train |>
    mutate(is_missing = is.na(.data[[col]])) |>
    group_by(is_missing) |>
    summarise(applicants   = n(),
              default_rate = mean(TARGET),
              .groups = "drop") |>
    mutate(column = col, .before = 1)
}

c("EXT_SOURCE_1", "EXT_SOURCE_3", "OWN_CAR_AGE",
  "COMMONAREA_AVG", "OCCUPATION_TYPE") |>
  map(missing_vs_target) |>
  list_rbind() |>
  knitr::kable(digits = 4,
               caption = "Default rate when a column is missing versus present")
```

Default rate when a column is missing versus present

column	is_missing	applicants	default_rate
EXT_SOURCE_1	FALSE	134133	0.0750
EXT_SOURCE_1	TRUE	173378	0.0852
EXT_SOURCE_3	FALSE	246546	0.0777

column	is_missing	applicants	default_rate
EXT_SOURCE_3	TRUE	60965	0.0931
OWN_CAR_AGE	FALSE	104582	0.0724
OWN_CAR_AGE	TRUE	202929	0.0850
COMMONAREA_AVG	FALSE	92646	0.0691
COMMONAREA_AVG	TRUE	214865	0.0857
OCCUPATION_TYPE	FALSE	211120	0.0879
OCCUPATION_TYPE	TRUE	96391	0.0651

Missingness here is not random, and that is the finding. `EXT_SOURCE_3` is missing for 19.8% of applicants and those applicants default at 9.3% against 7.8% for applicants who have a score. `COMMONAREA_AVG` shows the same direction, 8.6% against 6.9%. The mechanism is plausible: an applicant the external bureaus cannot score, or whose building is not in the reference database, is a thinner file overall, and thin files carry more risk.

`OCCUPATION_TYPE` runs the other way. Its 31.3% of missing rows default at 6.5% against 8.8% for rows with a value, because the missing group is dominated by pensioners who have no occupation to report and who repay well.

The consequence for data preparation is that **dropping rows with missing values would throw away the riskiest applicants and bias the model downward**. Imputation alone would also erase the signal. The plan is to impute and add a companion missing-indicator column for any predictor where the gap moves the default rate, so the model keeps both the filled value and the fact that it was filled.

4.3 Impossible and suspect values

```
# Each row is a value that cannot be literally true, or is extreme enough that
# an underwriter would refuse to believe it without documentation.
tibble(
  check = c("DAYS_EMPLOYED equal to 365243 (1,000 years of employment)",
            "CODE_GENDER coded XNA",
            "FLAG_MOBIL equal to 0",
            "AMT_INCOME_TOTAL above 10 million",
            "AMT_CREDIT greater than AMT_GOODS_PRICE"),
  rows  = c(sum(train$DAYS_EMPLOYED == 365243),
            sum(train$CODE_GENDER == "XNA"),
            sum(train$FLAG_MOBIL == 0),
            sum(train$AMT_INCOME_TOTAL > 1e7),
            sum(train$AMT_CREDIT > train$AMT_GOODS_PRICE, na.rm = TRUE))
) |>
  mutate(share = rows / nrow(train)) |>
  knitr::kable(digits = 4, caption = "Values that cannot be taken at face value")
```

Values that cannot be taken at face value

check	rows	share
DAYS_EMPLOYED equal to 365243 (1,000 years of employment)	55374	0.1801

check	rows	share
CODE_GENDER coded XNA	4	0.0000
FLAG_MOBIL equal to 0	1	0.0000
AMT_INCOME_TOTAL above 10 million	3	0.0000
AMT_CREDIT greater than AMT_GOODS_PRICE	198763	0.6464

```
# 365243 days is 1,000 years, so it is a sentinel rather than a measurement.
# Who are these applicants?
train |>
  filter(DAYS_EMPLOYED == 365243) |>
  count(NAME_INCOME_TYPE, sort = TRUE, name = "applicants") |>
  knitr::kable(caption = "Income type of applicants carrying the 365243 sentinel")
```

Income type of applicants carrying the 365243 sentinel

NAME_INCOME_TYPE	applicants
Pensioner	55352
Unemployed	22

`DAYS_EMPLOYED` is the serious one. **55,374 rows, 18% of the file, carry the value 365243**, which is 1,000 years. It is a placeholder, not a measurement, and 55,352 of those 55,374 applicants are pensioners. Left as a number it would wreck any employment-tenure feature, since the mean of the column comes out to 63,815 days. The fix is to recode 365243 to `NA` and add a flag for "not currently employed," which preserves the information without corrupting the scale.

The other four are smaller. Four `XNA` gender rows and one `FLAG_MOBIL` of zero are too rare to model and will be folded into the majority level or dropped. Three incomes above 10 million against a median of 147,150 look like data-entry errors and will be capped rather than deleted, since the rest of those applications look normal.

`AMT_CREDIT` exceeding `AMT_GOODS_PRICE` on **64.6% of loans** is not an error at all. Credit routinely exceeds the price of the goods because fees and insurance are financed into the loan. Worth flagging so nobody later treats loan-to-value above 1 as dirty data.

4.4 Keys, duplicates, and grain

```
# Confirm each table's key is unique and that no applicant appears in both the
# training and test files, which would leak the outcome.
tibble(
  check = c("Duplicate SK_ID_CURR in application_train",
            "Duplicate SK_ID_CURR in application_test",
            "Duplicate SK_ID_BUREAU in bureau",
            "SK_ID_CURR appearing in both train and test"),
  count = c(sum(duplicated(train$SK_ID_CURR)),
            sum(duplicated(test$SK_ID_CURR)),
            sum(duplicated(bureau$SK_ID_BUREAU)),
            length(intersect(train$SK_ID_CURR, test$SK_ID_CURR)))
```

```
) |>
  knitr::kable(caption = "Key and grain checks")
```

Key and grain checks

check	count
Duplicate SK_ID_CURR in application_train	0
Duplicate SK_ID_CURR in application_test	0
Duplicate SK_ID_BUREAU in bureau	0
SK_ID_CURR appearing in both train and test	0

Everything returns zero. The grain is exactly what the dictionary claims, the keys are clean, and train and test are disjoint populations. No deduplication is needed and there is no identifier leakage between the two files.

4.5 Temporal direction

```
# Every DAYS_ column is measured relative to the application date, so negative
# means before. A positive value means the record postdates the decision and
# could not have been known to an underwriter.
bureau |>
  summarise(
    DAYS_CREDIT           = sum(DAYS_CREDIT > 0, na.rm = TRUE),
    DAYS_CREDIT_UPDATE    = sum(DAYS_CREDIT_UPDATE > 0, na.rm = TRUE),
    DAYS_ENDDATE_FACT     = sum(DAYS_ENDDATE_FACT > 0, na.rm = TRUE),
    DAYS_CREDIT_ENDDATE   = sum(DAYS_CREDIT_ENDDATE > 0, na.rm = TRUE)
  ) |>
  pivot_longer(everything(), names_to = "column", values_to = "rows_after_application")
        |>
  mutate(share = rows_after_application / nrow(bureau)) |>
  knitr::kable(digits = 4, caption = "Bureau rows dated after the application")
```

Bureau rows dated after the application

column	rows_after_application	share
DAYS_CREDIT	0	0.0000
DAYS_CREDIT_UPDATE	17	0.0000
DAYS_ENDDATE_FACT	0	0.0000
DAYS_CREDIT_ENDDATE	602603	0.3511

This check matters because the business problem statement constrains the project to information available at the time of the lending decision. Three of the four columns are clean: `DAYS_CREDIT` and `DAYS_ENDDATE_FACT` have no positive values at all, and `DAYS_CREDIT_UPDATE` has 17 out of 1.7 million.

`DAYS_CREDIT_ENDDATE` is positive on **35.1% of bureau rows**, but that is legitimate rather than leakage. An open loan at application time has a scheduled end date in the future, and the underwriter knows it. The distinction is that a *scheduled* future date is knowable while an *observed* outcome is not, which is

why `DAYS_ENDDATE_FACT` having zero positives is the reassuring result. No bureau rows need to be filtered out.

4.6 Train and test consistency

```
# Which columns differ between the two files?
setdiff(names(train), names(test))
```

```
[1] "TARGET"
```

```
setdiff(names(test),  names(train))
```

```
character(0)
```

```
# A category present in train but never in test cannot be learned usefully, and
# a category present only in test will break a model that never saw it.
level_diffs <- train |>
  select(where(is.character)) |>
  names() |>
  map(function(col) {
    tibble(
      column     = col,
      train_only = paste(setdiff(na.omit(unique(train[[col]])),
                                 na.omit(unique(test[[col]]))), collapse = ", "),
      test_only  = paste(setdiff(na.omit(unique(test[[col]])),
                                 na.omit(unique(train[[col]]))), collapse = ", ")
    )
  }) |>
  list_rbind() |>
  filter(train_only != "" | test_only != "")

level_diffs |>
  knitr::kable(caption = "Categorical levels that appear in only one file")
```

Categorical levels that appear in only one file

column	train_only	test_only
CODE_GENDER	XNA	
NAME_INCOME_TYPE	Maternity leave	
NAME_FAMILY_STATUS	Unknown	

`TARGET` is the only column that differs, which is exactly right for a held-out prediction set. Three categorical levels appear in train and never in test: `CODE_GENDER` of XNA, `NAME_INCOME_TYPE` of “Maternity leave”, and `NAME_FAMILY_STATUS` of “Unknown”. Nothing appears only in test, which is the dangerous direction and it is clear.

These orphan levels are tiny and they matter for a mechanical reason rather than a statistical one. A factor level with no test representation produces an all-zero dummy column, which some model implementations reject outright. All three will be collapsed into the nearest common level during data preparation.

5 Exploratory Results

5.1 Q1. How much of the signal sits in the external credit scores?

```
# Correlate every numeric column with TARGET, then rank by absolute strength.
# SK_ID_CURR is dropped because an ID has no meaning as a number.
target_cors <- train |>
  select(where(is.numeric), -SK_ID_CURR) |>
  summarise(across(-TARGET, ~ cor(.x, TARGET, use = "complete.obs"))) |>
  pivot_longer(everything(), names_to = "column", values_to = "correlation") |>
  arrange(desc(abs(correlation)))

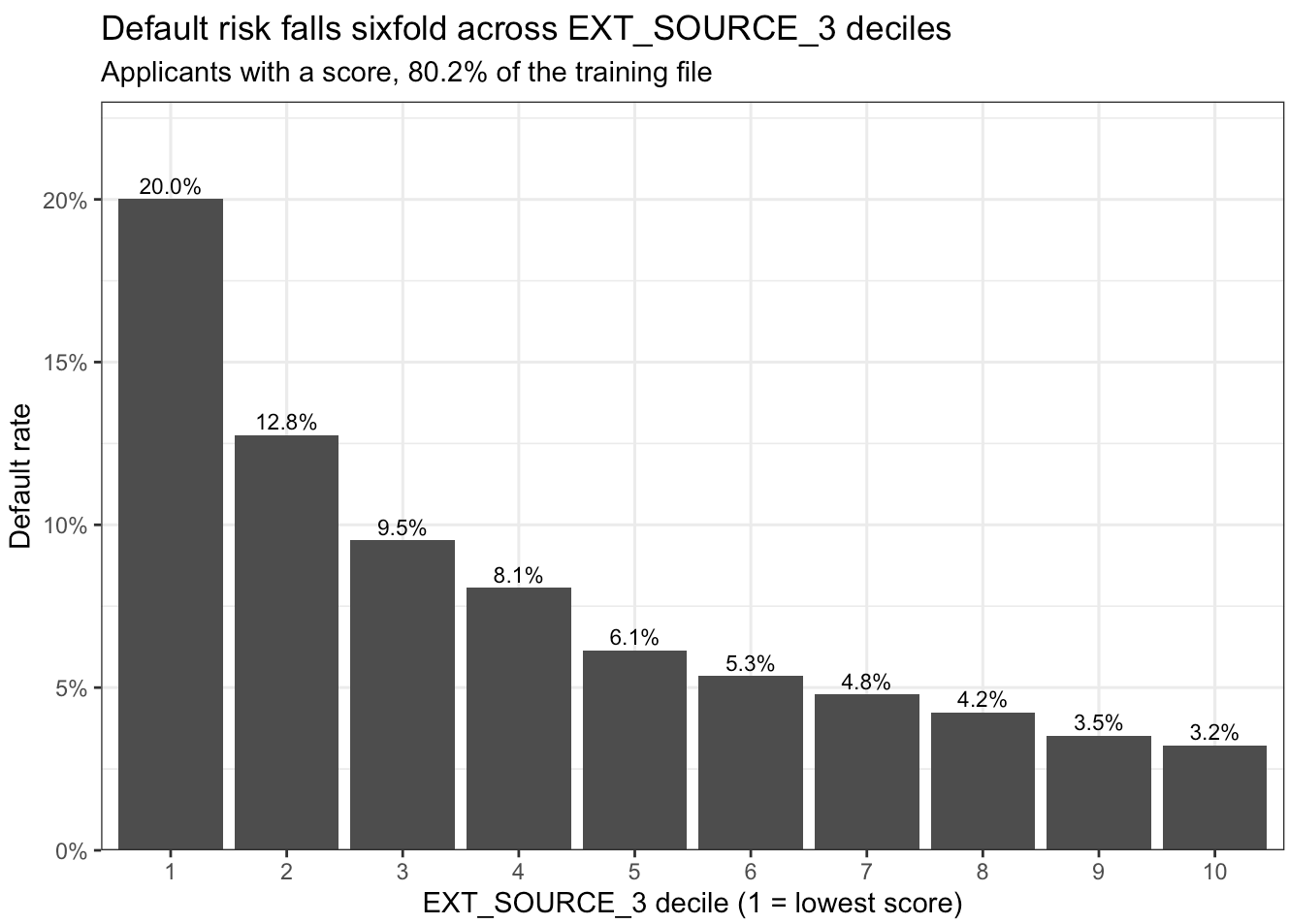
target_cors |>
  slice_head(n = 10) |>
  knitr::kable(digits = 4,
               caption = "Ten strongest linear relationships with the target")
```

Ten strongest linear relationships with the target

column	correlation
EXT_SOURCE_3	-0.1789
EXT_SOURCE_2	-0.1605
EXT_SOURCE_1	-0.1553
DAYS_BIRTH	0.0782
REGION_RATING_CLIENT_W_CITY	0.0609
REGION_RATING_CLIENT	0.0589
DAYS_LAST_PHONE_CHANGE	0.0552
DAYS_ID_PUBLISH	0.0515
REG_CITY_NOT_WORK_CITY	0.0510
FLAG_EMP_PHONE	0.0460

```
# ntile splits the scored applicants into ten equal groups. Plotting default
# rate by decile shows whether the relationship is monotonic, which a single
# correlation number cannot tell you.
train |>
  filter(!is.na(EXT_SOURCE_3)) |>
  mutate(decile = ntile(EXT_SOURCE_3, 10)) |>
  group_by(decile) |>
  summarise(default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = factor(decile), y = default_rate)) +
  geom_col(fill = "grey30") +
```

```
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Default risk falls sixfold across EXT_SOURCE_3 deciles",
       subtitle = "Applicants with a score, 80.2% of the training file",
       x = "EXT_SOURCE_3 decile (1 = lowest score)", y = "Default rate")
```




```
# The scores are powerful but incomplete. How often is each one available?
train |>
  summarise(across(starts_with("EXT_SOURCE"), ~ mean(is.na(.x)))) |>
  pivot_longer(everything(), names_to = "column", values_to = "pct_missing") |>
  knitr::kable(digits = 3, caption = "Availability of the three external scores")
```

Availability of the three external scores

column	pct_missing
EXT_SOURCE_1	0.564
EXT_SOURCE_2	0.002
EXT_SOURCE_3	0.198

The three external scores are the strongest columns in the file by a wide margin. `EXT_SOURCE_3` correlates with the target at -0.179, and the next best non-score column, `DAYS_BIRTH`, manages only

+0.078. The decile plot shows the relationship is clean and monotonic: the bottom decile defaults at 20.0% and the top at 3.2%, a sixfold spread with no reversals in between.

For the lender that is the whole ballgame, and it is also a problem. These columns are normalized scores from an outside vendor with no documentation of what goes into them. The business problem statement commits to decisions that can be explained to a declined applicant, and CFPB guidance is explicit that a complex model does not excuse a vague denial reason. “Your third-party score was low” is not a reason anyone can act on.

Availability compounds it. `EXT_SOURCE_1` is missing for 56.4% of applicants and `EXT_SOURCE_3` for 19.8%, so the strongest predictor is absent for exactly the thin-file population this business is built to serve. The approach this points to is a model that carries the scores but does not collapse onto them, and a parallel check on how performance holds up for applicants with no score at all.

5.2 Q2. Do applicants with no bureau history default at a different rate?

```
# Count bureau records per applicant, then join back. Applicants with no match
# get zero rather than NA, because absence of a record is the finding here.
bureau_counts <- bureau |>
  count(SK_ID_CURR, name = "bureau_credits")

train_bureau <- train |>
  left_join(bureau_counts, by = "SK_ID_CURR") |>
  mutate(bureau_credits = replace_na(bureau_credits, 0),
         history_depth  = cut(bureau_credits,
                              breaks = c(-1, 0, 2, 5, 10, Inf),
                              labels = c("None", "1 to 2", "3 to 5",
                                         "6 to 10", "11 or more")))

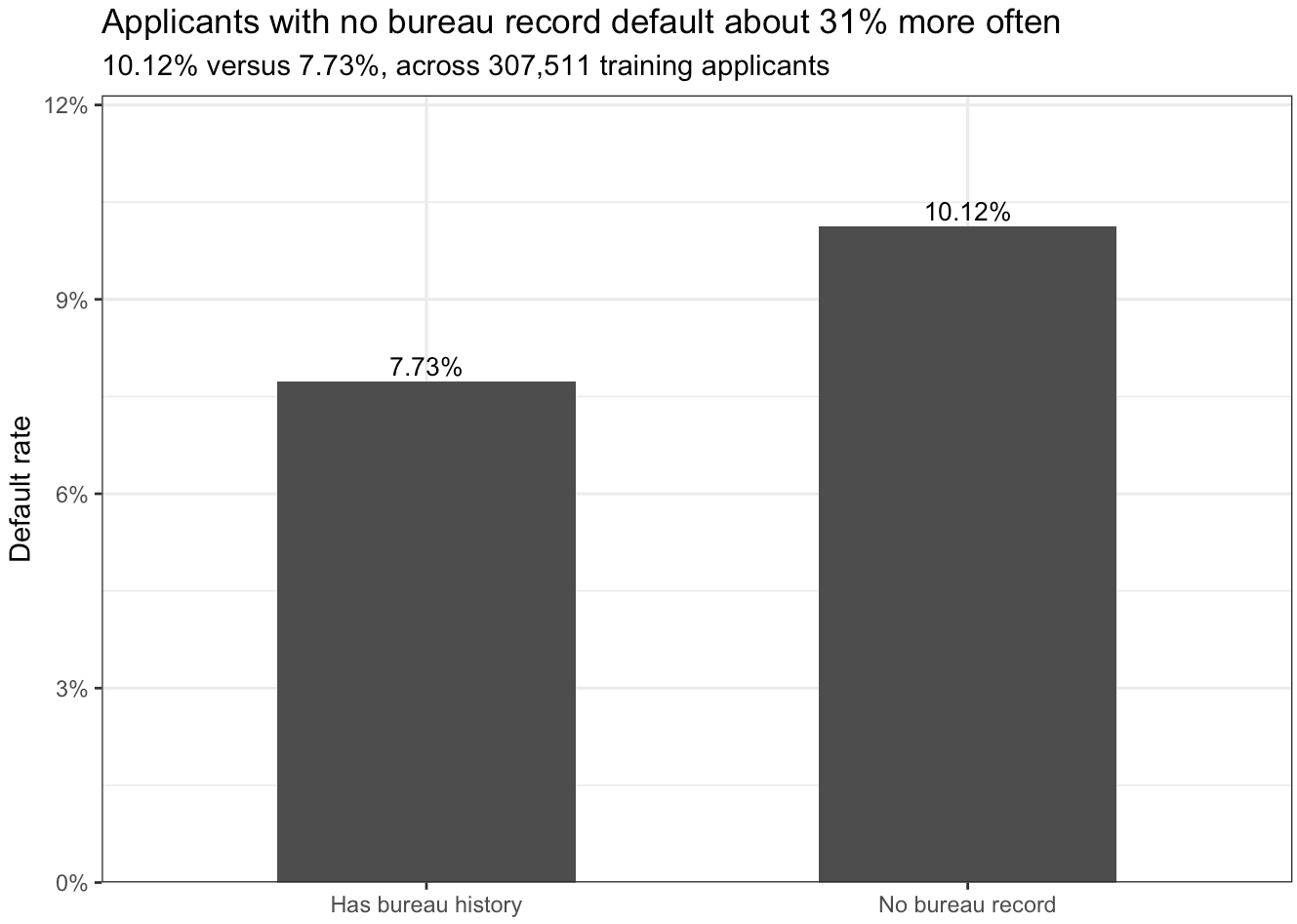
history_summary <- train_bureau |>
  group_by(history_depth) |>
  summarise(applicants   = n(),
            default_rate = mean(TARGET),
            .groups = "drop")

history_summary |>
  knitr::kable(digits = 4, caption = "Default rate by depth of credit bureau history")
```

Default rate by depth of credit bureau history

history_depth	applicants	default_rate
None	44020	0.1012
1 to 2	71707	0.0820
3 to 5	86883	0.0740
6 to 10	72624	0.0746
11 or more	32277	0.0818

```r
train_bureau |>
  mutate(has_history = if_else(bureau_credits > 0,
                               "Has bureau history", "No bureau record")) |>
  group_by(has_history) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = has_history, y = default_rate)) +
  geom_col(width = 0.55, fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.01)),
            vjust = -0.35, size = 3.5) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.2))) +
  labs(title = "Applicants with no bureau record default about 31% more often",
       subtitle = "10.12% versus 7.73%, across 307,511 training applicants",
       x = NULL, y = "Default rate")
```



This is the question the business problem statement rests on, and it holds up. The 14.3% of applicants with no bureau record default at 10.12% against 7.73% for applicants with history, roughly 31% higher in relative terms. Thin file genuinely does mean higher risk here, so the framing was correct.

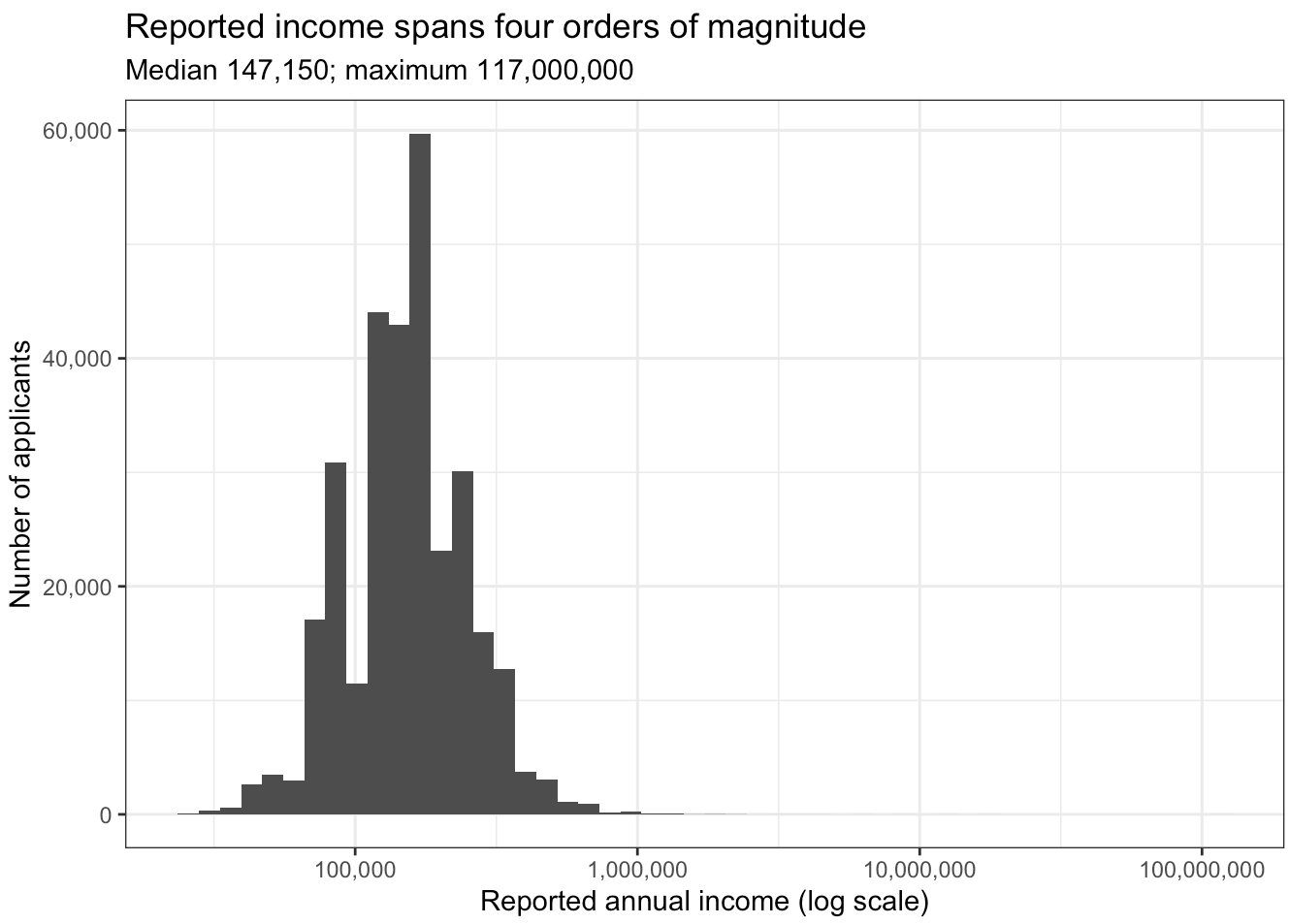
The depth breakdown adds something the binary split hides. Risk does not fall steadily as history deepens. It drops from 10.1% with no record to 7.4% at three to five credits, then **rises back to 8.2% for applicants carrying eleven or more**. The relationship is U-shaped. The mechanism is intuitive once

seen: a little credit history is evidence of reliability, while a lot of open credit lines is evidence of leverage or of shopping for credit under stress.

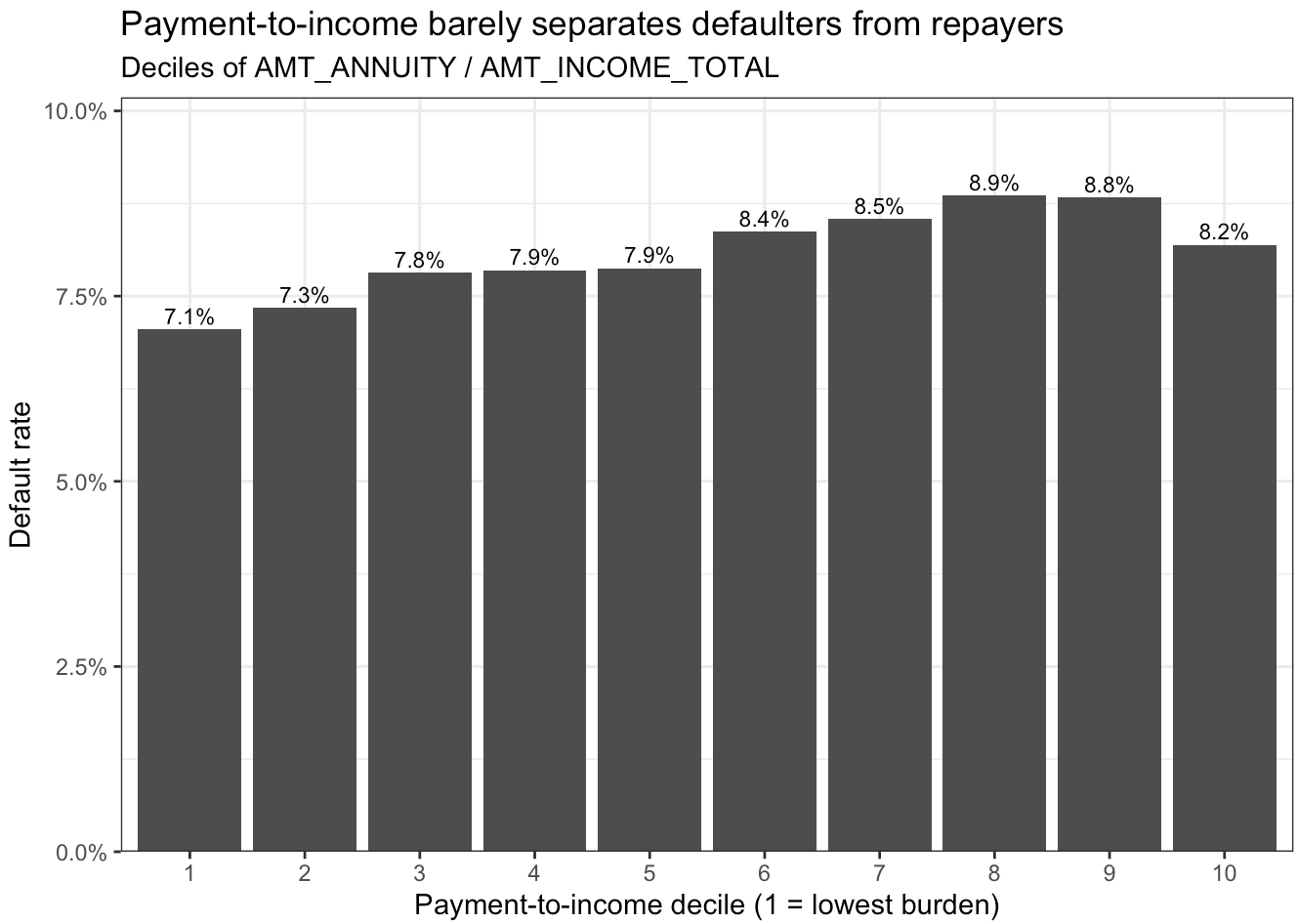
Two consequences for modeling. First, bureau record count has to enter as bands or a spline rather than a linear term, because a straight line through a U finds nothing. Second, “no bureau record” needs its own explicit flag rather than being imputed to zero, since zero and low are opposite ends of the risk curve.

5.3 Q3. Is income believable, and does payment-to-income separate defaulters?

```
# Income spans several orders of magnitude, so a log scale is the only way to
# see the distribution. The long right tail is the thing to look at.
train |>
  ggplot(aes(x = AMT_INCOME_TOTAL)) +
  geom_histogram(bins = 50, fill = "grey30") +
  scale_x_log10(labels = scales::comma) +
  scale_y_continuous(labels = scales::comma) +
  labs(title = "Reported income spans four orders of magnitude",
       subtitle = "Median 147,150; maximum 117,000,000",
       x = "Reported annual income (log scale)", y = "Number of applicants")
```



```r
# Payment-to-income is the standard ability-to-repay ratio: what share of
# income the loan payment consumes.
train |>
  mutate(pti = AMT_ANNUITY / AMT_INCOME_TOTAL) |>
  filter(!is.na(pti)) |>
  mutate(decile = ntile(pti, 10)) |>
  group_by(decile) |>
  summarise(default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = factor(decile), y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Payment-to-income barely separates defaulters from repayers",
       subtitle = "Deciles of AMT_ANNUITY / AMT_INCOME_TOTAL",
       x = "Payment-to-income decile (1 = lowest burden)", y = "Default rate")
```



Income is the weakest column in the file relative to how much underwriting theory says it should matter. Its correlation with the target is -0.004, which is nothing. On a log scale it improves only to -0.019.

The distribution explains part of why. The middle of the file is tight, with the 25th to 75th percentile running 112,500 to 202,500, but the tail is not credible. The 99th percentile is 472,500 and the maximum

is 117,000,000, roughly 250 times the 99th percentile. Two hundred fifty applicants report more than a million. This is self-reported, unverified income, and it behaves like it.

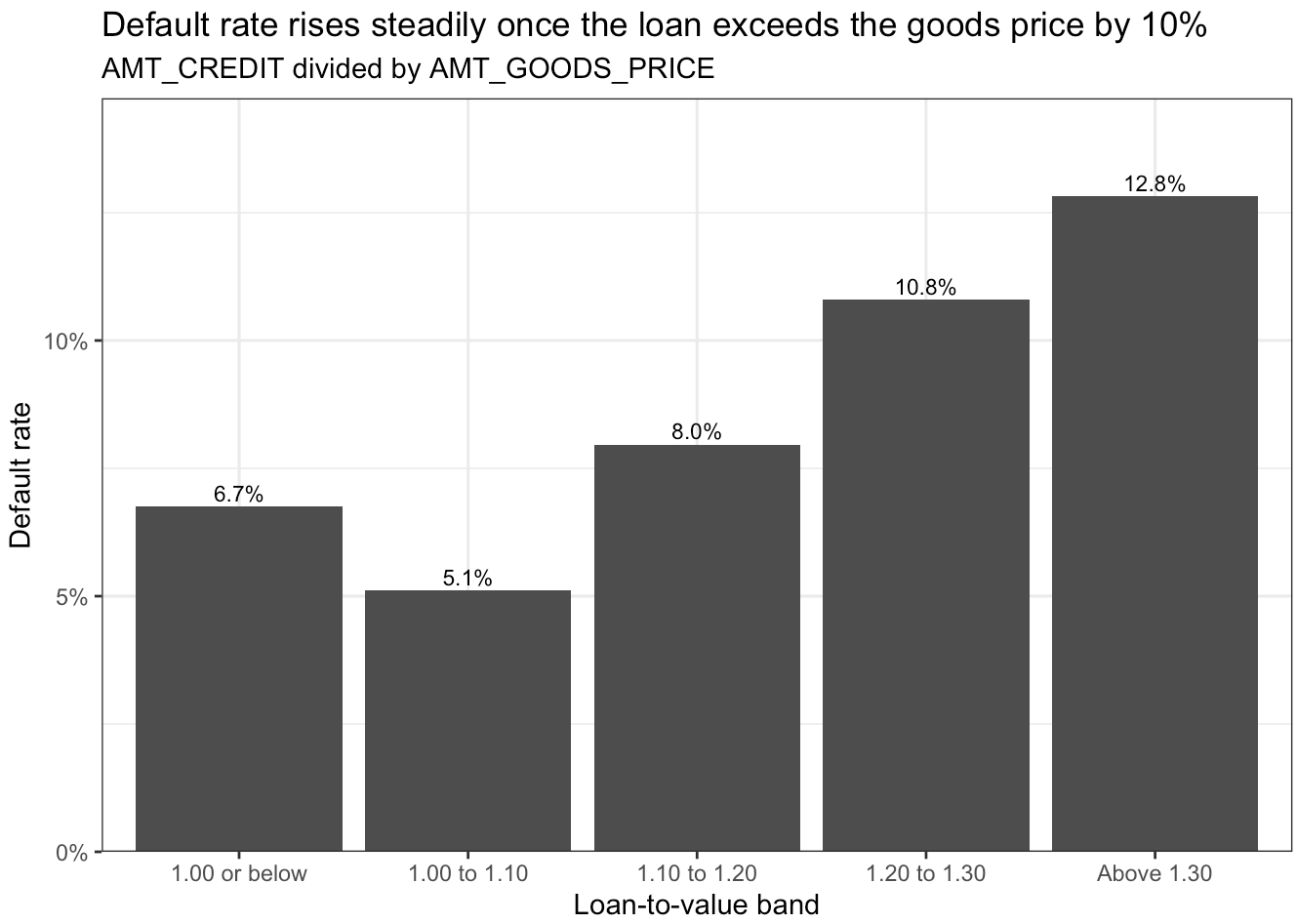
Payment-to-income, the ratio that ability-to-repay rules are built on, performs only slightly better. It climbs from 7.1% in the lowest burden decile to 8.9% in the ninth, then **falls back to 8.2% in the highest decile**, which is the opposite of what the theory predicts. The most burdened borrowers are not the riskiest.

The most likely mechanism is that Home Credit's existing underwriting already screens on affordability, so the highest-PTI loans that made it into this data are the ones that cleared some additional bar. That is survivorship in the sample, not a defect in the ratio.

Practically: PTI goes into the feature set because it is cheap, interpretable, and defensible to a regulator, but it will not carry the model. Raw income gets capped at the 99th percentile before use, and the expectation that income drives risk, which the business problem statement leans on implicitly, does not survive contact with this data.

5.4 Q4. Does loan-to-value relate to default?

```
# LTV is the loan amount divided by the price of the goods financed. Fixed
# bands are used instead of deciles because roughly a third of loans sit at
# exactly 1.00, and deciles would split that spike arbitrarily.
train |>
  mutate(ltv = AMT_CREDIT / AMT_GOODS_PRICE,
         ltv_band = cut(ltv,
                        breaks = c(0, 1, 1.1, 1.2, 1.3, Inf),
                        labels = c("1.00 or below", "1.00 to 1.10", "1.10 to 1.20",
                                   "1.20 to 1.30", "Above 1.30"))) |>
  filter(!is.na(ltv_band)) |>
  group_by(ltv_band) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = ltv_band, y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Default rate rises steadily once the loan exceeds the goods price by
        10%",
       subtitle = "AMT_CREDIT divided by AMT_GOODS_PRICE",
       x = "Loan-to-value band", y = "Default rate")
```



LTV carries real signal, and the shape is more interesting than a straight line. Loans at or below the goods price default at 6.8%, and loans between 1.00 and 1.10 default at only **5.1%, the safest band in the file**. Past 1.10 risk climbs without a break: 8.0%, then 10.8%, then 12.8% above 1.30. The spread from safest to riskiest band is two and a half times.

The mechanism is that the amount above the goods price is financed fees, insurance, and interest loaded into the principal. A small markup is routine product pricing. A large one means the borrower financed everything they could, which is a behavioral signal about their cash position at the moment of application, not a fact about the goods.

This makes LTV a strong candidate feature and, unlike the external scores, a fully explainable one. Note that it should enter as bands or a spline, because the dip at 1.00 to 1.10 means a linear term would misread the low end.

5.5 Q5. What is the DAYS_EMPLOYED sentinel, and should that group be separated?

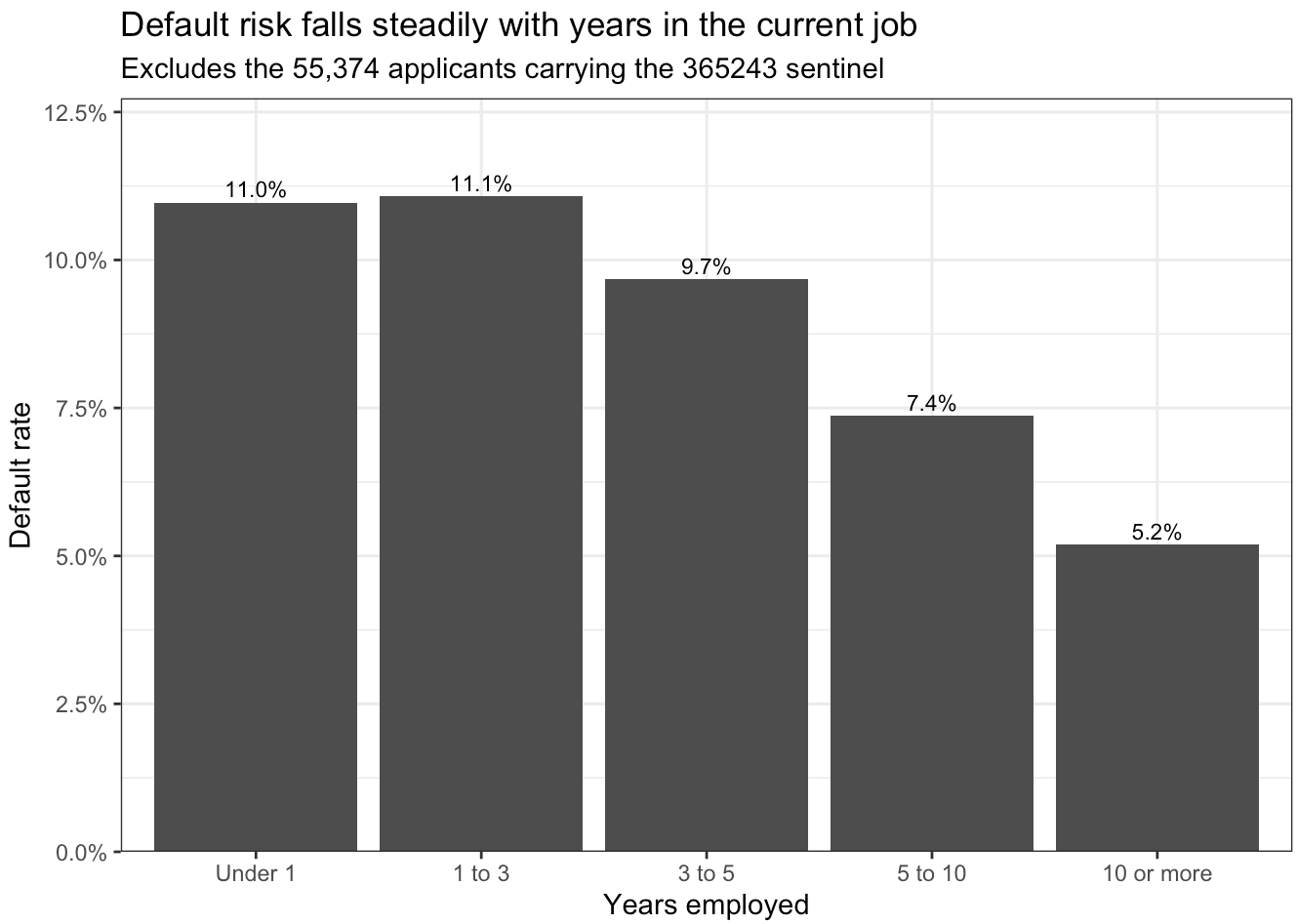
```
# Split the file on the sentinel value and compare outcomes directly.
train |>
  mutate(employment_recorded = if_else(DAYS_EMPLOYED == 365243,
                                       "Sentinel (365243)", "Real tenure value")) |>
```

```
  group_by(employment_recorded) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  knitr::kable(digits = 4,
               caption = "Outcomes for applicants carrying the employment sentinel")
```

Outcomes for applicants carrying the employment sentinel

employment_recorded	applicants	default_rate
Real tenure value	252137	0.0866
Sentinel (365243)	55374	0.0540

```
# For everyone with a real value, convert days to years and band it. Negative
# days mean days before the application, so the sign is flipped.
train |>
  filter(DAYS_EMPLOYED != 365243) |>
  mutate(years_employed = -DAYS_EMPLOYED / 365.25,
         tenure_band = cut(years_employed,
                           breaks = c(0, 1, 3, 5, 10, Inf),
                           labels = c("Under 1", "1 to 3", "3 to 5",
                                      "5 to 10", "10 or more"),
                           include.lowest = TRUE)) |>
  group_by(tenure_band) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = tenure_band, y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Default risk falls steadily with years in the current job",
       subtitle = "Excludes the 55,374 applicants carrying the 365243 sentinel",
       x = "Years employed", y = "Default rate")
```



Yes, the group should be separated, and the numbers say why. Applicants carrying the sentinel default at **5.40% against 8.66%** for everyone else. They are overwhelmingly pensioners, which means the sentinel is not missing information at all. It is a category, and a low-risk one, because pension income is stable and verifiable in a way wage income in these markets is not.

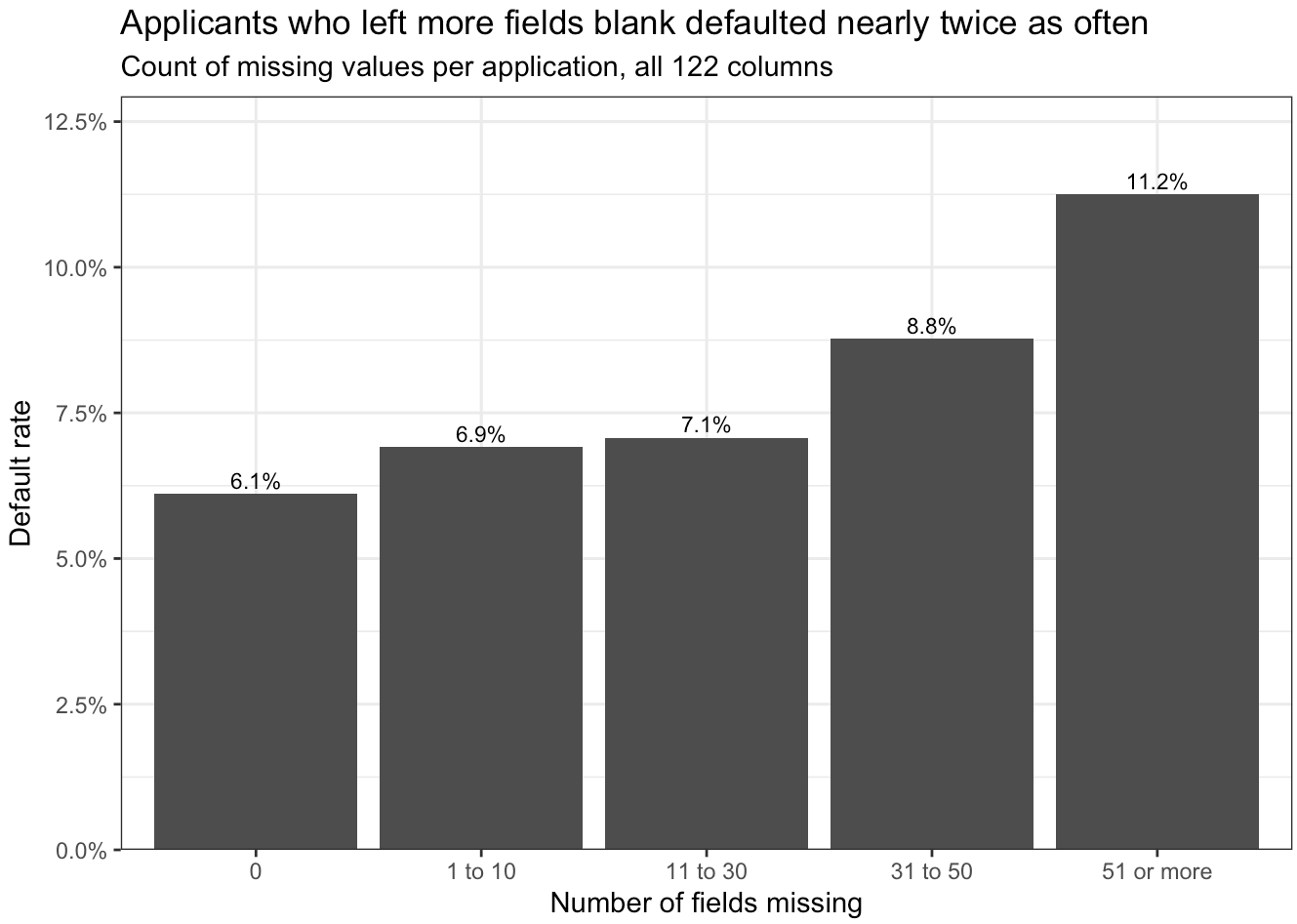
Once the sentinel rows are removed, employment tenure becomes one of the cleanest predictors available. Risk falls monotonically from 11.0% under one year to 5.2% at ten years or more, with no reversals.

Both facts get lost if the sentinel stays in as a number. It would drag the column mean to 63,815 days and flatten the real gradient into noise. Data preparation recodes 365243 to `NA`, adds a binary `not_currently_employed` flag, and keeps tenure as a continuous feature for everyone else.

5.6 Q6. Is missingness itself predictive?

```
# Rather than checking columns one at a time, count how many fields each
# applicant left blank and treat that count as a feature in its own right.
train |>
  mutate(fields_missing = rowSums(is.na(pick(everything()))),
         missing_band = cut(fields_missing,
                            breaks = c(-1, 0, 10, 30, 50, Inf),
```

```
                          labels = c("0", "1 to 10", "11 to 30",
                                     "31 to 50", "51 or more"))) |>
  group_by(missing_band) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = missing_band, y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "Applicants who left more fields blank defaulted nearly twice as often",
       subtitle = "Count of missing values per application, all 122 columns",
       x = "Number of fields missing", y = "Default rate")
```



The answer is yes, and it is one of the more useful findings in this EDA. Default rate climbs from 6.1% for applicants with a complete record to **11.2% for those missing 51 or more fields**, nearly double, and it rises at every step. The median applicant is missing 36 fields, so this is not an edge case.

The mechanism is that incompleteness is a behavioral signal, not a data defect. An applicant who supplies less is one the institution knows less about, and that is exactly the thin-file condition the business problem statement describes. Some of it is structural, since building columns are blank for anyone whose address is not in the reference database, but the gradient is smooth enough that the composite still works.

The action is concrete: create a `fields_missing` count feature and carry missing-indicator flags alongside imputed values. Imputing silently would discard a predictor that costs nothing to compute and is trivially explainable to a regulator, which the external scores are not.

5.7 Q7. Which application characteristics show the largest gaps, and are the segments large enough to act on?

```
# Summarise default rate by level for four categorical columns, keeping the
# segment size next to every rate so small-sample noise stays visible.
rate_by_level <- function(col) {
  train |>
    group_by(level = .data[[col]]) |>
    summarise(applicants   = n(),
              default_rate = mean(TARGET),
              .groups = "drop") |>
    mutate(column = col, .before = 1) |>
    arrange(desc(default_rate))
}

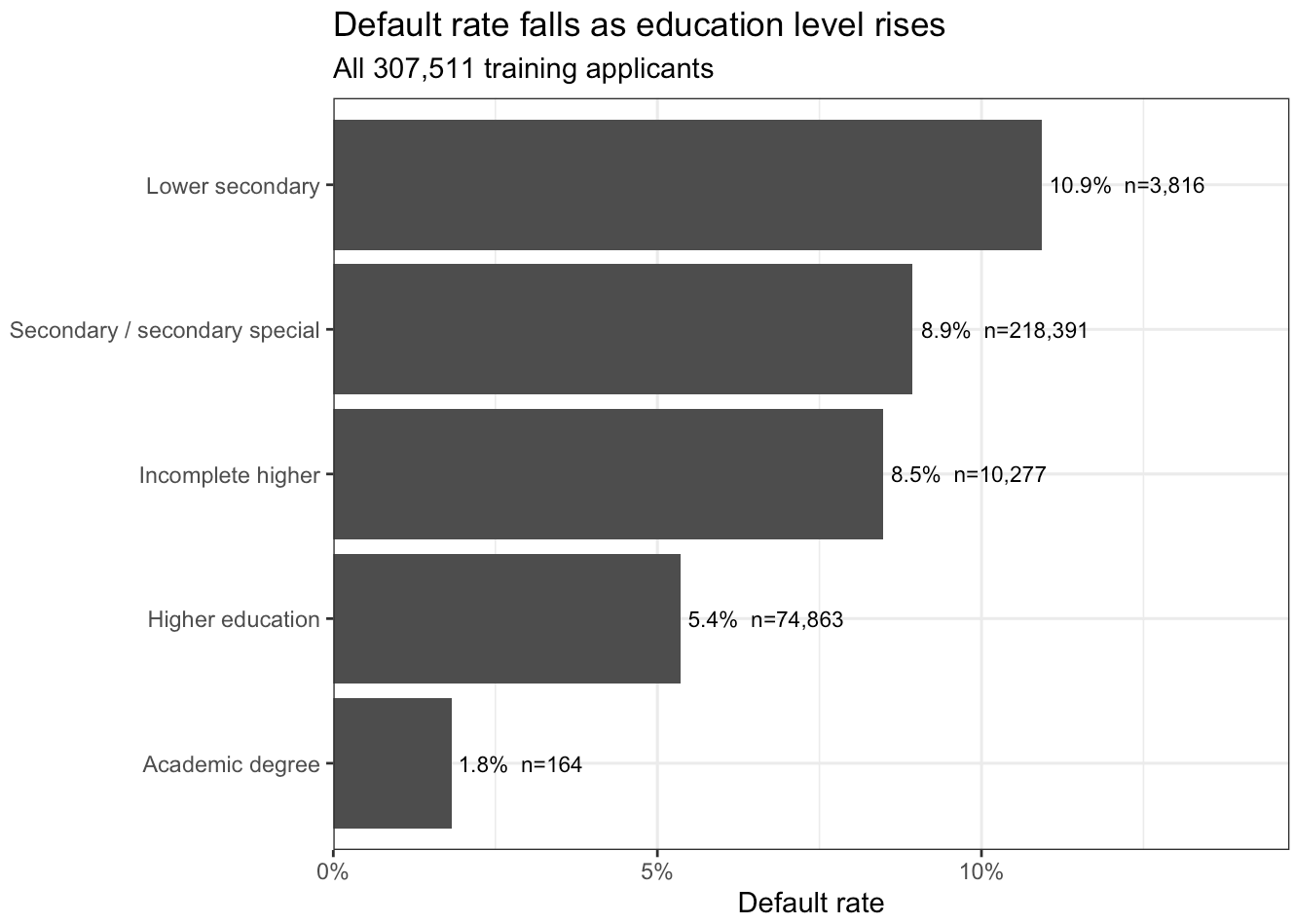
c("NAME_EDUCATION_TYPE", "NAME_INCOME_TYPE",
  "NAME_FAMILY_STATUS", "NAME_HOUSING_TYPE") |>
  map(rate_by_level) |>
  list_rbind() |>
  knitr::kable(digits = 4,
               caption = "Default rate and segment size by categorical level")
```

Default rate and segment size by categorical level

column	level	applicants	default_rate
NAME_EDUCATION_TYPE	Lower secondary	3816	0.1093
NAME_EDUCATION_TYPE	Secondary / secondary special	218391	0.0894
NAME_EDUCATION_TYPE	Incomplete higher	10277	0.0848
NAME_EDUCATION_TYPE	Higher education	74863	0.0536
NAME_EDUCATION_TYPE	Academic degree	164	0.0183
NAME_INCOME_TYPE	Maternity leave	5	0.4000
NAME_INCOME_TYPE	Unemployed	22	0.3636
NAME_INCOME_TYPE	Working	158774	0.0959
NAME_INCOME_TYPE	Commercial associate	71617	0.0748
NAME_INCOME_TYPE	State servant	21703	0.0575
NAME_INCOME_TYPE	Pensioner	55362	0.0539
NAME_INCOME_TYPE	Businessman	10	0.0000
NAME_INCOME_TYPE	Student	18	0.0000
NAME_FAMILY_STATUS	Civil marriage	29775	0.0994
NAME_FAMILY_STATUS	Single / not married	45444	0.0981
NAME_FAMILY_STATUS	Separated	19770	0.0819

column	level	applicants	default_rate
NAME_FAMILY_STATUS	Married	196432	0.0756
NAME_FAMILY_STATUS	Widow	16088	0.0582
NAME_FAMILY_STATUS	Unknown	2	0.0000
NAME_HOUSING_TYPE	Rented apartment	4881	0.1231
NAME_HOUSING_TYPE	With parents	14840	0.1170
NAME_HOUSING_TYPE	Municipal apartment	11183	0.0854
NAME_HOUSING_TYPE	Co-op apartment	1122	0.0793
NAME_HOUSING_TYPE	House / apartment	272868	0.0780
NAME_HOUSING_TYPE	Office apartment	2617	0.0657

```
# Education is the cleanest of the four, and every level is large enough to
# trust except Academic degree.
train |>
  group_by(NAME_EDUCATION_TYPE) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = fct_reorder(NAME_EDUCATION_TYPE, default_rate), y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = paste0(scales::percent(default_rate, accuracy = 0.1),
                               "  n=", scales::comma(applicants))),
            hjust = -0.05, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.35))) +
  coord_flip() +
  labs(title = "Default rate falls as education level rises",
       subtitle = "All 307,511 training applicants",
       x = NULL, y = "Default rate")
```



The largest gaps in the file are also the least usable, and that is the finding. `NAME_INCOME_TYPE` shows "Maternity leave" at a 40.0% default rate and "Unemployed" at 36.4%, both far above anything else. Those segments contain **5 and 22 applicants**. Two more defaults in either one would move the rate by ten points. These are noise wearing the costume of a strong effect, and a model that splits on them will overfit.

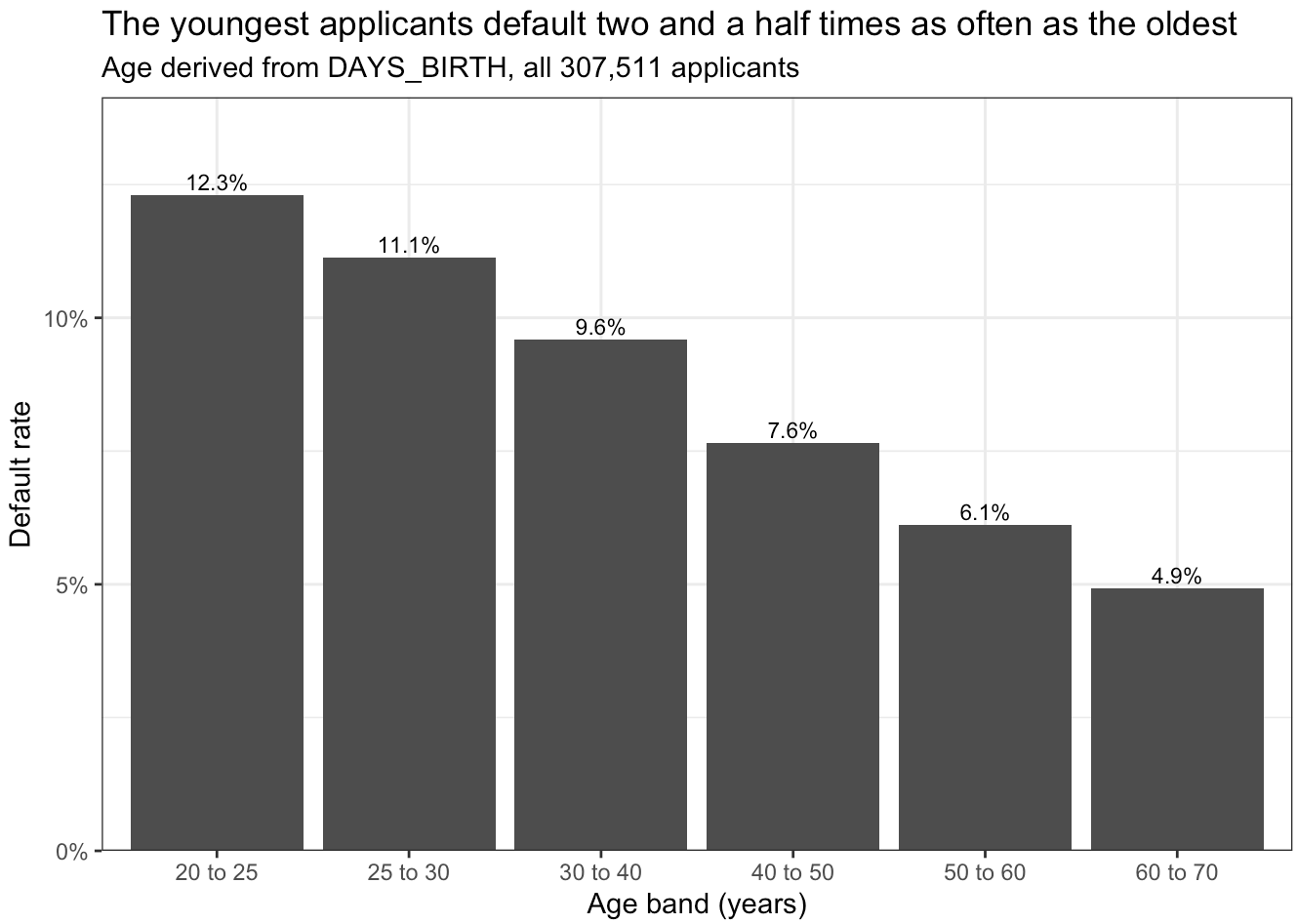
The gaps worth acting on are smaller and sit on real volume. Education runs from 10.9% for lower secondary down to 5.4% for higher education across 74,863 applicants, a clean monotonic gradient on segments large enough to trust. Housing type runs 12.3% for rented apartments and 11.7% for living with parents against 7.8% for owning a house or apartment, on 272,868 applicants. Family status separates civil marriage at 9.9% from married at 7.6%.

Two implications. First, rare levels get collapsed into an "Other" bucket during preparation rather than kept as their own dummies. Second, education and housing are strong, interpretable, and defensible, which makes them the kind of feature that can actually appear in an adverse action notice.

5.8 Q8. Does default risk change smoothly with age?

```
# DAYS_BIRTH is negative days since birth, so flipping the sign and dividing by
# 365.25 gives age in years.
train |>
```

```r
  mutate(age = -DAYS_BIRTH / 365.25,
         age_band = cut(age,
                        breaks = c(20, 25, 30, 40, 50, 60, 70),
                        labels = c("20 to 25", "25 to 30", "30 to 40",
                                   "40 to 50", "50 to 60", "60 to 70"),
                        include.lowest = TRUE)) |>
  group_by(age_band) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = age_band, y = default_rate)) +
  geom_col(fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.1)),
            vjust = -0.35, size = 3) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.15))) +
  labs(title = "The youngest applicants default two and a half times as often as the
        oldest",
       subtitle = "Age derived from DAYS_BIRTH, all 307,511 applicants",
       x = "Age band (years)", y = "Default rate")
```



Age behaves well. Risk falls monotonically from 12.3% among applicants aged 20 to 25 down to 4.9% among those aged 60 to 70, and every band holds at least 12,233 applicants so none of it is thin. Combined with the tenure gradient in Q5, the same underlying story shows up twice: stability, whether measured in years alive or years in a job, tracks repayment.

The relationship is close enough to linear that a continuous term would work, but the drop between the first two bands is steeper than the rest, so bands or a spline capture slightly more. Either is defensible.

One caution before this becomes a feature. Age is a protected characteristic under equal credit opportunity rules. A model can use it in some jurisdictions and not others, and even where permitted it invites scrutiny. The business problem statement already commits to a fairness and proxy-discrimination review, and age is the first column that review should look at. Employment tenure captures much of the same signal with far less regulatory exposure, which may make it the better choice despite the slightly weaker relationship.

5.9 Q9. Do recent credit bureau inquiries relate to default, and over what window?

```
# Six columns count enquiries to the credit bureau over widening windows, from
# the hour before application out to the past year. Check each one's
# availability and its raw relationship with the target.
inquiry_cols <- names(train)[str_detect(names(train), "^AMT_REQ_CREDIT_BUREAU")]

train |>
  summarise(across(all_of(inquiry_cols),
                   list(pct_missing = ~ mean(is.na(.x)),
                        correlation = ~ cor(.x, TARGET, use = "complete.obs")))) |>
  pivot_longer(everything(),
               names_to = c("column", "measure"),
               names_pattern = "(.*)_(pct_missing|correlation)") |>
  pivot_wider(names_from = measure, values_from = value) |>
  knitr::kable(digits = 4,
               caption = "Availability and target correlation of the six inquiry
      windows")
```

Availability and target correlation of the six inquiry windows

column	pct_missing	correlation
AMT_REQ_CREDIT_BUREAU_HOUR	0.135	0.0009
AMT_REQ_CREDIT_BUREAU_DAY	0.135	0.0027
AMT_REQ_CREDIT_BUREAU_WEEK	0.135	0.0008
AMT_REQ_CREDIT_BUREAU_MON	0.135	-0.0125
AMT_REQ_CREDIT_BUREAU_QRT	0.135	-0.0020
AMT_REQ_CREDIT_BUREAU_YEAR	0.135	0.0199

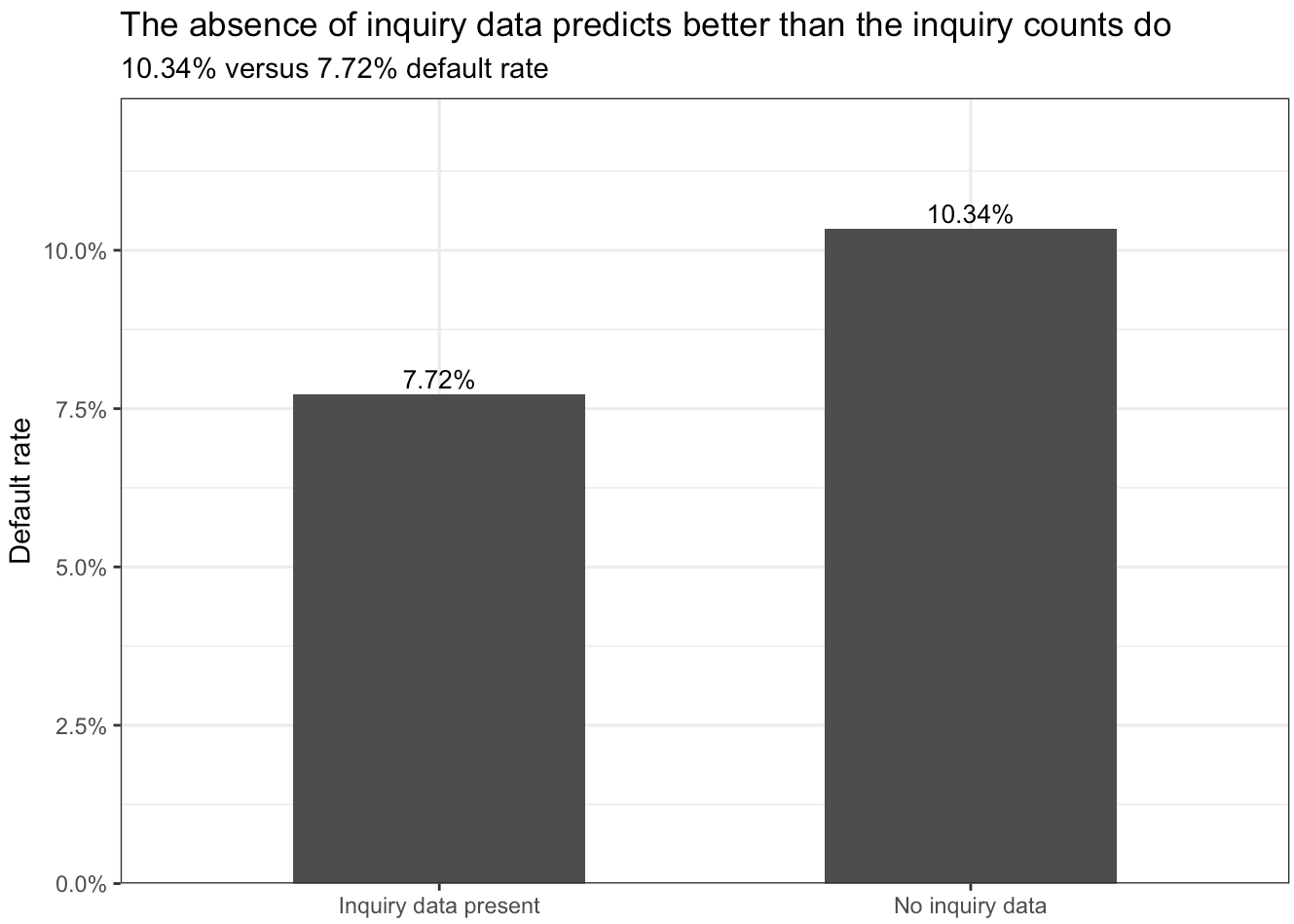
```
# The one-year window is the only one with enough volume to band meaningfully.
train |>
  filter(!is.na(AMT_REQ_CREDIT_BUREAU_YEAR)) |>
  mutate(inquiries = cut(AMT_REQ_CREDIT_BUREAU_YEAR,
                         breaks = c(-1, 0, 1, 2, 3, Inf),
                         labels = c("0", "1", "2", "3", "4 or more"))) |>
  group_by(inquiries) |>
```

```
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  knitr::kable(digits = 4,
               caption = "Default rate by bureau enquiries in the past year")
```

Default rate by bureau enquiries in the past year

inquiries	applicants	default_rate
0	71801	0.0713
1	63405	0.0733
2	50192	0.0810
3	33628	0.0796
4 or more	46966	0.0855

```
# The columns themselves are weak. But whether the inquiry block is populated
# at all turns out to matter, so test that directly.
train |>
  mutate(inquiry_record = if_else(
    rowSums(is.na(pick(all_of(inquiry_cols)))) == length(inquiry_cols),
    "No inquiry data", "Inquiry data present")) |>
  group_by(inquiry_record) |>
  summarise(applicants = n(), default_rate = mean(TARGET), .groups = "drop") |>
  ggplot(aes(x = inquiry_record, y = default_rate)) +
  geom_col(width = 0.55, fill = "grey30") +
  geom_text(aes(label = scales::percent(default_rate, accuracy = 0.01)),
            vjust = -0.35, size = 3.5) +
  scale_y_continuous(labels = scales::percent,
                     expand = expansion(mult = c(0, 0.2))) +
  labs(title = "The absence of inquiry data predicts better than the inquiry counts do",
       subtitle = "10.34% versus 7.72% default rate",
       x = NULL, y = "Default rate")
```



This one comes back negative, and the negative result is informative. Inquiry velocity is a well-established distress signal in consumer lending, so the expectation going in was that it would show up here. It does not. The strongest of the six windows is the one-year count at a correlation of **+0.0199**, and the others sit at essentially zero. The one-year bands do trend in the right direction, 7.1% at zero inquiries up to 8.6% at four or more, but that is a 1.5 point spread against the sixfold spread `EXT_SOURCE_3` produced.

The one-month window actually runs backwards, correlating at -0.0125, with applicants making four or more enquiries in the past month defaulting at 5.3% against 7.8% for those making none. The likeliest explanation is that short-window enquiries in this data reflect Home Credit's own processing of an active, engaged applicant rather than that applicant shopping desperately for credit elsewhere. A column that measures the lender's behavior rather than the borrower's will not predict the borrower's behavior.

The usable finding is in the third chunk. All six columns are missing together for the same **13.5% of applicants**, and those applicants default at 10.34% against 7.72%. That mirrors the thin-file result from Q2 almost exactly, which makes sense: if there is no bureau relationship, there is nothing to enquire against and nothing to record.

So the six raw counts are low-value features and the one-month version is close to actively misleading. What earns a place in the feature set is a single binary flag for whether bureau enquiry data exists at all, which is the same underlying signal Q2 and Q6 both surfaced from different directions.

6 Results and Conclusions

6.1 What the data turned out to be

`application_train` is 307,511 loan applications by 122 columns, one row per application, keyed cleanly on `SK_ID_CURR` with no duplicates and no overlap with the test file. The grain is exactly what the dictionary claims and the keys are trustworthy. Almost nothing else about the file is as clean as that.

The width is misleading. Of the 122 columns, 47 describe the apartment block the applicant lives in and 27 are contact and document flags. Only five describe the loan and only three carry external credit scores. The supplied dictionary misnames the bureau key, skips two rows, gives two different columns the same description, and hands all 47 building columns a single shared paragraph. It is a starting point, not an authority, and every column used here was verified against the data itself.

6.2 Data problems found, and the decision for each

Problem	Scale	Decision for data preparation
`DAYS_EMPLOYED` sentinel of 365243	55,374 rows, 18.0%	Recode to `NA`, add a `not_currently_employed` flag. Left alone it drags the column mean to 63,815 days
Missing values are informative, not random	67 of 122 columns, worst at 69.9%	Impute, but carry a missing-indicator beside every predictor whose gap moves the default rate. Never drop rows
Reported income is not credible in the tail	Max 117,000,000 against a 147,150 median	Cap at the 99th percentile of 472,500. Do not delete the rows
Rare categorical levels with no test representation	`XNA` gender (4), Maternity leave (5), Unemployed (22), Unknown status (2)	Collapse into an "Other" bucket. All-zero dummies break some model implementations
No bureau record at all	44,020 applicants, 14.3%	Explicit flag. Do not impute to zero, since zero and low sit at opposite ends of the risk curve
`AMT_CREDIT` exceeds `AMT_GOODS_PRICE`	198,763 loans, 64.6%	Not an error. Fees are financed into principal. Document it so nobody later "cleans" it
Bureau enquiry columns are weak or reversed	Six columns, best correlation +0.0199	Drop the raw counts. Keep one binary flag for whether enquiry data exists
Building columns arrive as AVG, MODE, MEDI triplets	47 columns	Strong redundancy is likely from the naming structure alone, but it was not tested here. Flagged as the first dimensionality check before modeling

Temporal direction is clean. `DAYS_ENDDATE_FACT` has zero values dated after the application and `DAYS_CREDIT_UPDATE` has 17 out of 1.7 million, so no bureau rows need to be filtered. The 35.1% of `DAYS_CREDIT_ENDDATE` values that fall after the application date are scheduled end dates on open loans, which an underwriter knows at decision time and which are therefore not leakage.

6.3 The strongest relationships with the target

Ranked by the spread between the safest and riskiest group:

Predictor	Safest group	Riskiest group	Spread
`EXT_SOURCE_3` (deciles)	3.2%	20.0%	6.3x
Loan-to-value band	5.1%	12.8%	2.5x
Age band	4.9%	12.3%	2.5x
Employment tenure	5.2%	11.0%	2.1x
Education level	5.4%	10.9%	2.0x
Housing type	6.6%	12.3%	1.9x
Count of missing fields	6.1%	11.2%	1.8x
Bureau record present	7.7%	10.1%	1.3x

And the ones that did not work: payment-to-income moved default rate only from 7.1% to 8.9% and then fell back in the top decile. Raw income correlates with the target at -0.004. Bureau enquiry counts are near zero across all six windows and the one-month version runs backwards.

6.4 How this changed the analytics approach

Four assumptions from the business problem statement held up. The 8.07% base rate is confirmed, which justifies ROC AUC over accuracy and means the model needs class weighting or a tuned threshold rather than a 0.5 cutoff. Thin-file applicants really are riskier, at 10.12% against 7.73%. The data really is historical and anonymized. And the decision-time constraint is satisfiable, because the temporal checks came back clean.

One assumption did not survive. The business problem statement leans on affordability, and affordability is the thing this data cannot measure. Income is self-reported and unverified, its correlation with default is essentially zero, and payment-to-income is nearly as weak. The likely reason is that the existing underwriting process already screens on affordability, so what remains in the data is the population that already cleared that bar. Payment-to-income stays in the feature set because it is cheap and defensible to a regulator, but the project cannot be built on it.

One relationship is more complicated than assumed. Credit history depth is U-shaped, not linear. Default falls from 10.1% with no bureau record to 7.4% at three to five credits, then climbs back to 8.2% at eleven or more. Some history signals reliability; a lot of it signals leverage. Any bureau count feature has to be banded or splined, because a linear term through a U finds nothing.

One finding was not anticipated at all. Missingness is a predictor. Applicants who left 51 or more fields blank default at 11.2% against 6.1% for those with a complete record, and the gradient rises at every step. This costs nothing to compute, is trivially explainable in an adverse action notice, and would have been destroyed by a standard impute-and-move-on pipeline.

The central tension is now explicit. The three strongest columns in the file are external scores with no documented derivation, and `EXT_SOURCE_1` is missing for 56.4% of applicants. So the best signal is both the least explainable and thinnest exactly where the business needs it most. The business problem

statement commits to decisions that can be explained to a declined applicant, and CFPB guidance is clear that model complexity does not excuse a vague denial reason.

The approach that follows is to build two models rather than one. A full model carrying the external scores establishes the performance ceiling. A restricted model built only on interpretable, well-populated features (loan-to-value, employment tenure, education, housing, missingness count, bureau record flag) establishes what is actually deployable. The gap between their AUCs is the real decision for leadership: what accuracy the business gives up in exchange for a score it can defend. That comparison was not in the original analytics approach and it should be.

One item for the fairness review the business problem statement already promises. Age is the second strongest usable predictor and it is a protected characteristic. Employment tenure captures much of the same underlying stability signal with far less regulatory exposure, and may be the better choice even though its relationship is slightly weaker. Region ratings and address-mismatch flags need the same scrutiny as geographic proxies.

Appendix: Use of AI

I used Claude throughout this notebook, following the stage-gated workflow from the course slides rather than asking it to "do EDA."

The stages. I ran five. First, a data map: grain, keys, join paths, and any place the dictionary contradicted the data, with no analysis allowed. Second, industry research on what consumer lenders actually use for repayment risk, with sources restricted to regulator guidance and peer-reviewed work rather than Kaggle notebooks. Third, I wrote the question list myself from my business problem statement, the data map, and that research. Fourth, execution, one question at a time. Fifth, a critique pass where I asked it to argue against my own conclusions.

What worked well. Separating the stages mattered more than I expected. Asking for a data map before any analysis surfaced the dictionary problems immediately, including that the bureau key is documented as `SK_BUREAU_ID` when the actual column is `SK_ID_BUREAU`. That mistake broke a script on contact, which was a useful early lesson that the dictionary could not be trusted anywhere.

Writing the questions myself was the part that made the notebook mine. When I asked for a draft question list, what came back was reasonable but generic. Cutting it, reordering it, and tying each question back to a specific claim in my business problem statement is what gave Section 4 a spine.

What did not work. My first instinct was to ask for large chunks at once, and the output was technically correct but far beyond what I needed. I got better results by requesting one section at a time and rejecting anything that introduced concepts I had not asked for. Two of my early prompts produced code with paths and setup assumptions that did not match my actual project layout, which cost time to debug. I find that going section by section allows me to follow along and be collaborative with AI in the EDA process.

Verification. Every number in this notebook was computed twice. Claude ran the calculations independently against the same CSVs and reported what each chunk should produce before I ran it, so a

mismatch meant something was wrong. That caught real errors rather than just producing agreeable output.

Where I overrode it. The negative results are the clearest case. Claude's first framing of the bureau enquiry section treated a weak positive correlation as a finding. The honest read is that the enquiry columns do not work here, and that the one-month window runs backwards because it is probably measuring Home Credit's own processing rather than the applicant's behavior. Reporting that as a failed hypothesis is more useful than dressing up a 0.02 correlation. Same with payment-to-income, where the interesting result is that the textbook ratio underperforms and why.

Overall. AI made the code essentially free, which moved all of the difficulty to deciding what to ask and what the answers meant. That matches what the course slides predicted. The notebook is better than what I would have produced alone in the time available, but only because I treated it as an analyst I had to direct and check rather than as an oracle.